Lista de Exercícios de Fixação — Aula 03

Encapsulamento, Propriedades e a Tela do Sistema

Continue com o projeto escolhido na Aula 02

Reaproveite o mesmo projeto escolhido na Aula 02 — é importante que seja a mesma classe que você já escreveu, porque hoje o exercício é uma **refatoração** dela, não uma criação do zero. Se você não tiver a classe da Aula 02 em mãos pode recriá-la rapidamente com atributos públicos antes de começar (é rápido, e serve até de revisão). Qualquer dúvida leia o arquivo de **Exercícios Aula 02**.

Como referência rápida, a mesma tabela da Aula 02:

#	Projeto	Entidade A	Entidade B
1	CineClub	Filme	Cliente
2	VetCare	Tutor	Animal
3	StockShop	Fornecedor	Produto
4	FitControl	Plano	Aluno
5	TaskFlow	Colaborador	Tarefa
6	CyberTime	Máquina	Cliente
7	LinguaMax	Turma	Aluno
8	MesaFácil	Item do Cardápio	Mesa
9	PortalVisit	Morador	Visitante
10	GolPro	Time	Jogador
11	HotelEasy	Quarto	Hóspede
12	FarmaSys	Medicamento	Cliente
13	OficinaPro	Cliente	Veículo
14	EvenTicket	Evento	Participante
15	BikeShare	Bicicleta	Cliente
16	Escolinha	Modalidade	Aluno

#	Projeto	Entidade A	Entidade B
17	CondomínioNet	Unidade	Morador
18	TravelLog	Pacote	Cliente
19	SalaShow	Sala	Solicitante
20	PetShopPlus	Tutor	Animal

Exercício 1 — Réplica guiada (encapsulando a Entidade A)

Usando a classe da **Entidade A** feita na Aula 02:

1. Renomeie cada atributo público, tornando-o `private` e prefixado com `_` (ex.: `Title` → `_title`).
2. Crie uma propriedade `public` para cada um, com `get` e `set`, usando o mesmo nome que o atributo tinha antes de virar privado (ex.: a propriedade continua se chamando `Title`).
3. Ajuste os dois construtores (com e sem parâmetros) para atribuírem através das propriedades (`this.Title = title;`), e não mais diretamente aos atributos privados.
4. Rode novamente o `Main` que você já tinha da Aula 02, **sem alterar nenhuma linha dele**. Se compilar e imprimir exatamente igual a antes, a refatoração está correta.

Critério de correção: todo atributo virou privado com `_`; toda propriedade tem `get` e `set`; o `Main` da Aula 02 continua funcionando sem edição.

Exercício 2 — Encapsulando a Entidade B

Repita o Exercício 1 para a classe da **Entidade B** do mesmo projeto (a que foi feita no Exercício 2 da lista da Aula 02).

Exercício 3 — "Isso mereceria uma validação?"

Sem alterar o código da sua classe (ela continua com `set` simples, sem `if` nenhum), escolha **dois atributos** da Entidade A do seu projeto e, para cada um, escreva um comentário no código respondendo:

1. Que valor **inválido** alguém poderia tentar colocar nesse atributo (ex.: uma capacidade negativa, um preço zerado, uma data de nascimento no futuro)?

2. Se essa verificação existisse, ela ficaria dentro do `set` da propriedade (no `Model`) ou em outro lugar do sistema? Justifique com uma frase, usando a mesma lógica discutida em aula sobre o `BookModel`.

Exercício 4 — Testando a `Tela`, sem ligação com o projeto ainda

Independente do projeto sorteado, todo aluno cria e testa sua própria classe `Tela.cs`, idêntica à construída em aula (`MontarMoldura`, `LimparArea`, `Centralizar`, `Perguntar`).

No `Main`, instancie um objeto `Tela` e:

1. Desenhe uma moldura em qualquer posição da tela.
2. Centralize, dentro dela, um título que combine com o **nome do seu projeto** (ex.: "Cadastro de Bicicletas", para quem pegou o BikeShare).
3. Use `Perguntar` para pedir alguma informação simples ao usuário (pode ser qualquer pergunta de teste, sem relação com o cadastro de verdade ainda) e exiba a resposta na tela.

Critério de correção: a moldura aparece corretamente fechada (os quatro cantos e as quatro bordas), o título aparece centralizado (não colado na borda esquerda nem cortado), e `Perguntar` devolve a resposta digitada.

Exercício 5 — Encontre o erro

Trecho A — uma propriedade:

```
public string Nome
{
    get { return _nome; }
    set { _nome = _nome; }
}
```

Trecho B — uma chamada de `MontarMoldura`:

```
Tela tela = new Tela();
tela.MontarMoldura(40, 5, 10, 15);
```